MANUAL DE INSTALACIÓN LOCAL

FaceLit · Aplicación web/móvil, API, servicio facial y base de datos

FaceLit	Backend	micro-facial	FaceLit-DB
Expo · Web/Android	Spring Boot · 8080	FastAPI · 8090	PostgreSQL · 5439 -> 5432

Propósito y alcance

Este documento prepara un entorno **local de desarrollo** con los cuatro repositorios FaceLit. Resume requisitos, configuración y el orden de arranque verificado en los archivos actuales del proyecto. No es una guía para instalar o desplegar en el VPS.

Flujo principal: la interfaz Expo consume la API Spring Boot; el backend usa PostgreSQL y delega operaciones biométricas al servicio FastAPI mediante una conexión interna y una clave compartida. Liquibase administra el esquema.

Antes de comenzar

- Trabaja en ramas y revisiones compatibles entre frontend, backend, servicio facial y base de datos.
- Usa datos ficticios para pruebas; no copies secretos ni datos biométricos reales a capturas o documentos.
- Conserva cualquier archivo .env existente. Crea uno desde .env.example solo cuando no exista.
- Los comandos de Liquibase update cambian el esquema. Ejecutarlos únicamente contra la base local prevista.

1. Requisitos y preparación

Instala las herramientas requeridas y abre PowerShell. Verifica en cada README las instrucciones de la rama que vas a usar.

Componente	Requisito de referencia	Uso
Frontend FaceLit	Node.js/npm compatibles con package-lock; Android SDK para build nativo	Web, Expo/Android
FaceLit-Backend	Java 17 y Maven Wrapper incluido	API en 8080
micro-facial	Docker recomendado; imagen basada en Python 3.11	FastAPI en 8090
FaceLit-DB	Docker Desktop con Compose	PostgreSQL 16/pgvector; host 5439

Obtener o actualizar los repositorios

Usa las URL de GitHub configuradas para el equipo. No mezcles archivos .env locales con cambios de rama; confirma que los cuatro componentes corresponden a la misma entrega.

```
git clone <URL-del-repositorio>
# Repositorios: FaceLit, FaceLit-Backend, micro-facial, FaceLit-DB
# Si ya existen: git status; git pull --ff-only
```

Mapa de puertos

- **5439** en el equipo → **5432** dentro del contenedor de PostgreSQL.
- **8090** servicio facial; **8080** backend; el puerto de Expo/Metro lo asigna su herramienta.

Con teléfono físico, localhost apunta al teléfono. Configura una IP o nombre LAN que el dispositivo pueda alcanzar.

2. Base de datos y migraciones

Inicia PostgreSQL antes de los otros servicios. El Compose de FaceLit-DB crea la red Docker **facelit-docker-compose_default**, que necesita el Compose independiente de micro-facial.

En PowerShell, desde la carpeta FaceLit-DB

```
docker compose -p facelit-docker-compose up -d postgres
docker compose -p facelit-docker-compose ps
docker compose -p facelit-docker-compose --profile tooling build liquibase
docker compose -p facelit-docker-compose --profile tooling run --rm liquibase validate
docker compose -p facelit-docker-compose --profile tooling run --rm liquibase status
docker compose -p facelit-docker-compose --profile tooling run --rm liquibase update-sql
```

Revisa la salida de update-sql. Si los cambios son los esperados y el destino es la base local, aplica las migraciones:

```
docker compose -p facelit-docker-compose --profile tooling run --rm liquibase update
```

El servicio de base usa PostgreSQL 16 con pgvector. El backend arranca con validación Hibernate y Liquibase deshabilitado; por eso, el esquema debe migrarse previamente. No uses update contra producción.

Conexión desde procesos locales

Los procesos ejecutados fuera de Docker deben usar el puerto publicado del host (normalmente 5439). Dentro de la red Docker el host de base es **postgres:5432**. Los valores exactos de base, usuario y contraseña se toman de la configuración local y nunca deben copiarse a este manual.

Comprobación de orden

- Base saludable y puerto local disponible.
- Liquibase valida; status/update-sql no muestran cambios inesperados.
- Luego se inicia el servicio facial y, después, backend y frontend.

3. Servicio facial (FastAPI)

El Compose de micro-facial conserva el modelo descargado en un volumen e inicia la API en el puerto 8090. Requiere que el repositorio FaceLit-DB ya haya creado la red externa.

Desde la carpeta micro-facial

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
# Edita .env localmente: DATABASE_URL, API_KEY y ajustes requeridos
docker compose build embedding-service
docker compose up -d embedding-service
docker compose logs --tail=80 embedding-service
```

La variable **DATABASE_URL** debe apuntar a la base prevista y **API_KEY** debe coincidir con la credencial privada del backend. El ejemplo del repositorio es una plantilla; configura credenciales propias. No publicar la clave.

Opción de ejecución nativa

En Windows, las dependencias de visión por computador pueden requerir herramientas C++ y librerías nativas; se recomienda el contenedor oficial del proyecto. Si ejecutas Python localmente, usa el intérprete Python 3.11 documentado:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --port 8090
```

El primer inicio puede tardar por la descarga y carga del modelo. La API expone /health; una respuesta de salud no sustituye la comprobación de una operación facial real. No utilices tests/support/e2e_app.py como entrypoint de la aplicación.

Red y acceso

- El Compose del servicio se conecta a la red externa facelit-docker-compose_default.
- Dentro de Docker, PostgreSQL se resuelve como postgres; desde Windows usa localhost y el puerto publicado.
- Mantén el servicio facial accesible solo para los componentes que lo requieren.

4. Backend y frontend

Backend FaceLit-Backend

Usa Java 17. El Maven Wrapper del repositorio descarga la versión de Maven necesaria. Configura la conexión JDBC y secretos mediante el mecanismo local del proyecto. Variables de referencia:

- DB_URL, DB_USERNAME, DB_PASSWORD
- JWT_SECRET
- FACIAL_EMBEDDING_URL y FACIAL_EMBEDDING_API_KEY
- FACELIT_ACCESS_URL; MAIL_USERNAME y MAIL_PASSWORD solo si se prueba correo.

```
.\mvnw.cmd spring-boot:run
```

El servicio escucha normalmente en 8080. Para acceso facial, la URL interna y la clave deben apuntar al microservicio del mismo entorno. El correo depende de SMTP configurado; no se considera verificado por el solo arranque.

Frontend FaceLit

Usa Node.js/npm compatibles con el lockfile actual. Configura **EXPO_PUBLIC_API_URL** con una dirección alcanzable desde el navegador o dispositivo. Las variables EXPO_PUBLIC se incluyen en el bundle y no deben contener credenciales privadas.

```
npm ci
npm run web
# Alternativas disponibles en package.json:
npm run start:lan
npm run android
```

Reinicia Expo después de cambiar la configuración. Para un móvil físico, localhost no corresponde al computador; usa una dirección LAN. Android nativo requiere el toolchain correspondiente. Expo Go y los túneles son herramientas de desarrollo; no son el servidor de producción.

Nota de compatibilidad

El README frontend aún documenta variables EXPO_PUBLIC_EMBEDDING_API_URL y EXPO_PUBLIC_EMBEDDING_API_KEY heredadas, mientras que el flujo facial actual revisado pasa por el backend. Confirma requisitos del build de la rama exacta y nunca incluyas claves privadas en EXPO_PUBLIC.

5. Verificación y solución de problemas

Arranque sugerido: PostgreSQL → migraciones → servicio facial → backend → frontend. Comprueba los logs de cada proceso antes de avanzar.

Lista de comprobación

- PostgreSQL saludable, pgvector disponible y puerto 5439 accesible desde el host.
- Liquibase validate correcto y migraciones aplicadas al entorno local previsto.
- FastAPI inicia con su configuración y modelo; /health responde.
- Backend inicia y valida el esquema existente; no crea tablas automáticamente.
- Frontend resuelve la URL API desde el tipo de dispositivo que se está usando.
- Demuestra login y un flujo con cuentas/datos ficticios. Evalúa cámara, correo y persistencia por separado.

Problemas frecuentes

Síntoma	Revisar
Backend no inicia	Variables DB_*, conexión PostgreSQL, puerto y cambios Liquibase requeridos por esta versión.
Microservicio no inicia	Red externa, archivo .env, clave, acceso a PostgreSQL, dependencias y descarga/carga de modelo.
Frontend no conecta	EXPO_PUBLIC_API_URL, firewall, IP LAN y que API y dispositivo puedan comunicarse. Reinicia Expo tras cambiar variables.
Falla facial pero login funciona	URL/clave backend→FastAPI, logs de ambos servicios, modelo y disponibilidad de la base.
Cámara web no disponible	Permiso del navegador, cámara y HTTPS según el entorno.
Migración inesperada	Detén update, revisa update-sql y confirma que DATABASE/URL corresponde a la instancia local.

Detener el entorno

```
docker compose down
```

Ejecuta el comando dentro del repositorio/Compose correspondiente. No agregues -v salvo que quieras borrar los volúmenes y los datos locales. Para cerrar los servicios, detén primero frontend/backend y luego servicios Docker.

Referencia documental: README y archivos Compose actuales de FaceLit, FaceLit-Backend, micro-facial y FaceLit-DB. Corte del manual: 27 de octubre de 2026. Verifica comandos y versiones contra los SHA de la entrega antes de distribuirlo.